

ONLINE LIBRARY MANAGEMENT SYSTEM

A Course Project Report

Submitted in the partial fulfilment of the completion of the course

23CS002PC304 AI Assisted Coding

In III Year II Semester B.Tech – CSE

Submitted by

B. Tharun

2303A51232

Under the supervision of the faculty

Sallauddin Md

Assistant Professor, SOCS & AI

Academic Year 2025-2026



Department of Computer Science & Engineering

SR University

Warangal, Telangana, India

INDEX

S.No	Section	Page No.
1	Abstract	3
2	Introduction	4
3	Literature Review	5
4	Problem Statement	6
5	Methodology / Procedure	7-8
6	System Design	9
7	Code	10-12
8	Output	13
9	Output Screenshots	14
10	Conclusion	15
11	References	16

1. ABSTRACT

The Online Library System is a comprehensive web-based application developed using the MERN stack, which includes MongoDB, Express.js, React.js, and Node.js. The system is designed to modernize and automate traditional library processes such as book cataloging, user management, and transaction handling. It provides an interactive and user-friendly interface that allows users to search for books, view availability, borrow or return books, and manage their profiles efficiently.

The application also provides an administrative panel that enables librarians to manage book inventories, track issued books, monitor user activities, and generate reports. By digitizing the library system, this project reduces manual errors, improves operational efficiency, and enhances accessibility for users.

The system ensures data consistency, security, and scalability by using modern web technologies and RESTful APIs. It is designed to handle multiple users simultaneously and can be extended to include advanced features such as recommendation systems and digital content access.

2. INTRODUCTION

Libraries have traditionally been centers of learning and knowledge dissemination. However, manual library systems often suffer from inefficiencies such as difficulty in maintaining records, tracking book availability, and managing user interactions. These challenges become more significant as the size of the library grows.

With the rapid advancement in technology, there is a growing need to adopt digital solutions that can streamline library operations. The Online Library System is developed to overcome these challenges by providing a centralized platform for managing all library-related activities.

Objectives of the System

- To automate library operations
- To provide easy access to books and resources
- To reduce manual workload
- To improve accuracy and efficiency
- To enhance user experience

Scope of the Project

The system is designed for:

- Educational institutions
- Public libraries
- Digital libraries

It supports both administrators and users, making it a scalable and flexible solution.

3. LITERATURE REVIEW

Various research studies and existing systems have contributed to the development of library management solutions.

3.1 Traditional Systems

Earlier library systems were manual, involving physical registers and records. These systems had several drawbacks:

- Time-consuming processes
- High chances of data loss
- Lack of real-time updates

3.2 Digital Library Systems

With the introduction of computers, digital systems were developed. These systems improved efficiency but often lacked:

- User-friendly interfaces
- Remote accessibility
- Scalability

3.3 Modern Web-Based Systems

Modern systems use full-stack technologies such as MERN, which provide:

- Real-time data synchronization
- Responsive design
- Cloud integration
- Secure authentication

3.4 Comparison with Existing Systems

Feature	Traditional	Old Digital	MERN-Based System
Accessibility	Low	Medium	High
Scalability	Low	Medium	High
Efficiency	Low	Medium	High

This project improves upon existing systems by integrating modern technologies and providing a seamless user experience.

4. PROBLEM STATEMENT

The existing library systems face multiple limitations that affect their efficiency and usability.

4.1 Key Problems

1. Manual data entry and record maintenance
2. Difficulty in tracking issued and available books
3. Limited accessibility for users
4. Lack of real-time updates
5. Inefficient search mechanisms
6. Poor user experience

4.2 Need for Solution

There is a need for a system that:

- Automates all library operations
- Provides real-time updates
- Ensures data security
- Improves accessibility and usability

The Online Library System aims to address these issues effectively.

5. METHODOLOGY / PROCEDURE

The existing library systems face multiple limitations that affect their efficiency and usability.

5.1 Key Problems

1. Manual data entry and record maintenance
2. Difficulty in tracking issued and available books
3. Limited accessibility for users
4. Lack of real-time updates
5. Inefficient search mechanisms
6. Poor user experience

5.2 Need for Solution

There is a need for a system that:

- Automates all library operations
- Provides real-time updates
- Ensures data security
- Improves accessibility and usability

The Online Library System aims to address these issues effectively.

5.3 AI Prompt Engineering Approach

The development process relied on seven carefully crafted prompts to Claude AI, each addressing a specific aspect of the application:

Prompt	Description
Prompt 1	Create a registration page using HTML, CSS, and JavaScript with form validation. Build a login and signup system with password strength validation.
Prompt 2	Create a responsive dashboard UI using HTML, CSS, and JavaScript. Design a modern navigation bar with dropdown menu and animations. Build a responsive landing page with header, footer, and sections.
Prompt 3	Create a contact form with name, email, and message validation. Build a student registration form with proper input validation. Create a feedback form with rating system and validation.
Prompt 4	Create a to-do list application with add, delete, and edit features. Build a calculator using HTML, CSS, and JavaScript. Create a search bar with live filtering functionality.
Prompt 5	Create a Node.js Express server for user authentication. Build REST API for CRUD operations using Express and MongoDB. Implement JWT authentication in Node.js.
Prompt 6	Create a complete library management system with frontend and backend. Build a full-stack web application with login and dashboard. Develop a student management system with CRUD operations.
Prompt 7	Design a responsive dashboard interface with navigation and user-specific features.

6. SYSTEM DESIGN

The Online Library System is designed using a layered **MVC (Model-View-Controller) architecture**, ensuring a clear separation between user interface, application logic, and data management. This architecture improves maintainability, scalability, and code organization.

6.1 Architecture Overview

Layer	Components & Role
Presentation	React.js, HTML5, CSS3, Bootstrap — renders user interface and handles user interactions
Controller	authController, userController, bookController, transactionController — manages business logic and request handling
Routes	authRoutes.js, userRoutes.js, bookRoutes.js, transactionRoutes.js — maps HTTP requests to controllers
Middleware	authMiddleware (JWT authentication), roleMiddleware (admin/user access), errorHandler — handles cross-cutting concerns
Model	User.js, Book.js, Transaction.js (Mongoose schemas) — defines data structure and validation
Database	MongoDB — stores users, books, borrowing records, and system data

6.3 Application Workflow

The workflow of the Online Library System begins when a user accesses the application through the browser.

Unauthenticated users are redirected to the **login or registration page**. After successful authentication, a **JWT token/session** is generated, and the user is granted access based on their role.

- **Admin Users** are redirected to the **Admin Dashboard**, where they can:
 - Add, update, or delete books
 - Manage users
 - View all transactions
 - Monitor system activity

- **Normal Users** are directed to the **User Dashboard**, where they can:
 - Search and view available books
 - Borrow books
 - Return books
 - View their borrowing history

All user requests are sent from the React frontend to the backend via API calls. The backend processes these requests using controllers, interacts with the MongoDB database, and sends responses back to the frontend.

7. CODE

The complete project consists of 44 files generated through AI prompting. Key source files are presented below:

7.1 admin.html— Main Application Entry Point

```
head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Admin Dashboard - AI Library</title>
<link rel="stylesheet" href="modern-style.css">
<style>
  .dashboard-container {
    display: grid;
    grid-template-columns: 250px 1fr;
    min-height: 100vh;
    background: #f8fafc;
  }

  .sidebar {
    background: linear-gradient(135deg, #6366f1 0%, #8b5cf6 100%);
    color: white;
    padding: 2rem 1.5rem;
    box-shadow: 0 10px 30px rgba(99, 102, 241, 0.2);
  }

  .sidebar-brand {
    font-size: 1.5rem;
    font-weight: 700;
    margin-bottom: 2rem;
    display: flex;
    align-items: center;
    gap: 0.5rem;
  }
```

```
.sidebar-menu {
  list-style: none;
}

.sidebar-link {
  color: rgba(255, 255, 255, 0.8);
  padding: 0.75rem 1rem;
  border-radius: 0.5rem;
  margin-bottom: 0.5rem;
  cursor: pointer;
  transition: all 0.3s ease;
  font-weight: 500;
}

.sidebar-link:hover,
.sidebar-link.active {
  background: rgba(255, 255, 255, 0.2);
  color: white;
}

.main-content {
  padding: 2rem;
  overflow-y: auto;
}

.dashboard-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 2rem;
}

.dashboard-header h1 {
  font-size: 2rem;
  color: #0f172a;
}

.logout-btn {
  padding: 0.5rem 1rem;
  background: #f43f5e;
  color: white;
  border: none;
  border-radius: 0.5rem;
  cursor: pointer;
  font-weight: 600;
  transition: all 0.3s ease;
}

.logout-btn:hover {
```

```

    transform: translateY(-2px);
    box-shadow: 0 5px 15px rgba(244, 63, 94, 0.3);
  }

  .content-section {
    background: white;
    border-radius: 1rem;
    padding: 2rem;
    margin-bottom: 2rem;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.07);
  }

  .section-title {
    font-size: 1.5rem;
    color: #0f172a;
    margin-bottom: 1.5rem;
    display: flex;
    align-items: center;
    gap: 0.5rem;
  }

```

7.2 models/books.js — User Schema

```

const mongoose = require('mongoose');

const bookSchema = new mongoose.Schema({
  title: String,
  author: String,
  category: String,
  stock: Number
});

module.exports = mongoose.model('Book', bookSchema);

```

7.3 models/issue.js — Food Schema

```

const mongoose = require('mongoose');

const issueSchema = new mongoose.Schema({
  studentId: String,
  bookId: String,
  fromDate: Date,
  toDate: Date,
  returned: {

```

```
    type: Boolean,  
    default: false  
  }  
});  
  
module.exports = mongoose.model('Issue', issueSchema);
```

7.4 models/user.js — Order Schema

```
const mongoose = require('mongoose');  
  
const userSchema = new mongoose.Schema({  
  username: String,  
  email: String,  
  password: String,  
  enroll: String,  
  role: String, //  comma added  
  otp: String,  
  otpExpiry: Date  
});  
  
module.exports = mongoose.model('User', userSchema);
```

8. OUTPUT

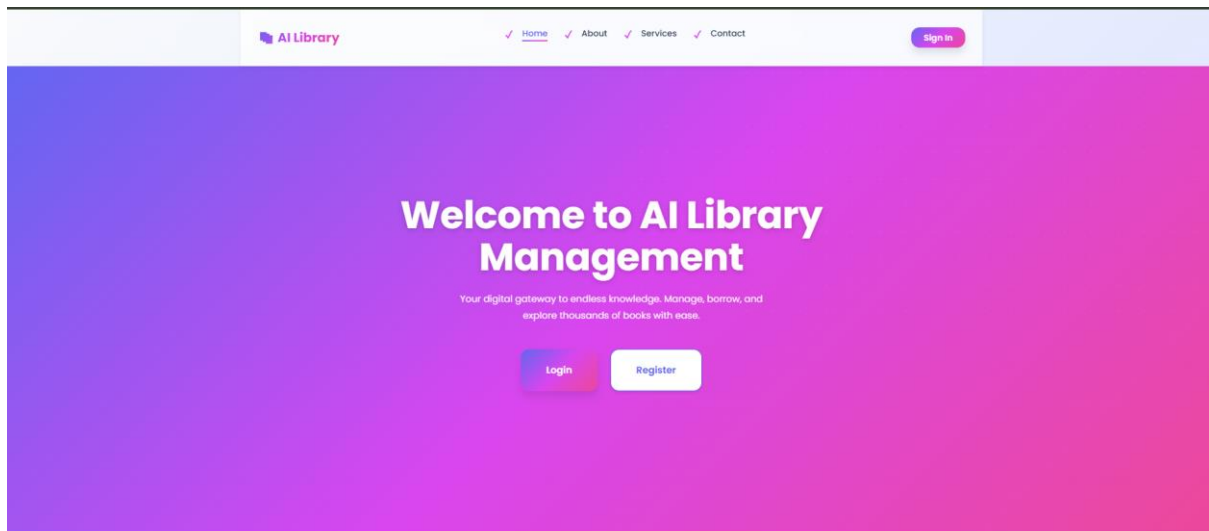
The following terminal output demonstrates a successful application startup and connection to MongoDB:

```
C:\Users\THARUN\OneDrive\Desktop\sem_3_2\ai\Ai_library_management>node  
server.js  
 Server running on http://localhost:3000  
 MongoDB Connected-- Static files served from /public --
```

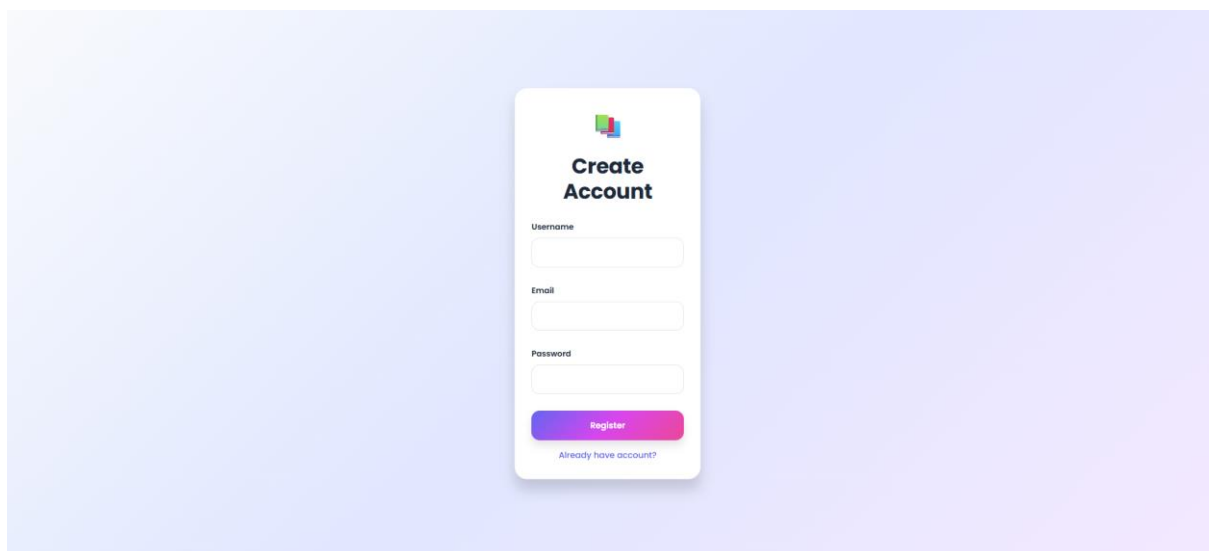
9. OUTPUT SCREENSHOTS

The following screenshots demonstrate the key views and functionality of the library application:

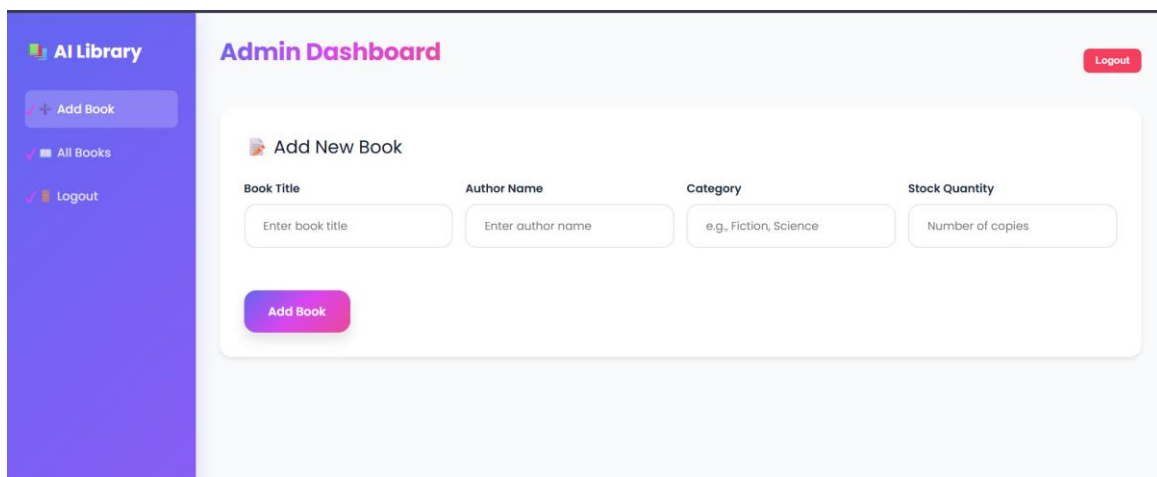
9.1 Login Page



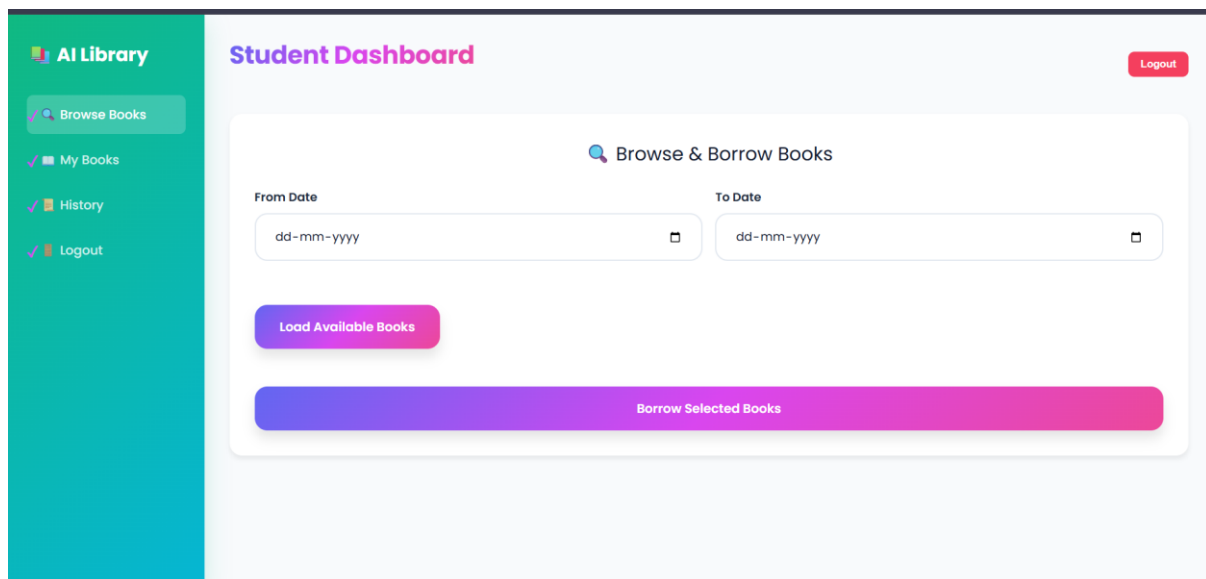
9.2 Registration Page



9.3 Admin Page



9.3 User Page



10. CONCLUSION

The Online Library System demonstrates the effective use of modern web technologies to solve real-world problems. By automating library operations, the system reduces manual effort and improves efficiency. The MERN stack provides a powerful platform for building scalable and high-performance applications. The system is user-friendly, secure, and adaptable to future enhancements.

The Online Library System developed using the MERN stack provides an efficient and modern solution to the limitations of traditional library management. By automating key operations such as book management, user registration, and borrowing/return processes, the system significantly reduces manual workload and minimizes errors. The use of a web-based platform ensures that users can access library services anytime and from anywhere, improving overall accessibility and convenience.

The implementation of a structured MVC architecture and integration of technologies like React.js, Node.js, Express.js, and MongoDB ensures that the system is scalable, maintainable, and performance-oriented. Features such as secure authentication, role-based access control, and real-time data handling enhance both security and usability. The system also provides a user-friendly interface that simplifies interactions for both administrators and users, making it suitable for real-world deployment in educational institutions and organizations.

Future Enhancements

- Integration with mobile applications
- AI-based recommendation system
- E-book and digital content support
- Barcode/QR code integration

11. REFERENCES

1. MongoDB Inc., *MongoDB Documentation*. Available at: <https://www.mongodb.com/docs/>
2. Meta Platforms Inc., *React.js Official Documentation*. Available at: <https://react.dev/>
3. OpenJS Foundation, *Node.js Documentation*. Available at: <https://nodejs.org/en/docs/>
4. OpenJS Foundation, *Express.js Guide*. Available at: <https://expressjs.com/>
5. Mozilla Developer Network (MDN), *Web Development Documentation*. Available at: <https://developer.mozilla.org/>